

MLOPS Lab 1 Exercises

g1.prayogihala-bits-pilani.ac.in/web-rdp/#/client/MTY3MjBjAHBvc3RncmVzcWw=token=H9fSA20B1Z53fDCfC9B3DA39D6319508CTD9D936885C75CF4A00C3A7E7E3C6

Applications MLflow — Mozilla Firefox cartvista-mlops - Thunar generate_data.py - cart...

MLflow x +

127.0.0.1:5000/#/experiments/1?searchFilter=&orderByKey=attributes.start_time&orderByAsc=false&startTime=ALL&lifecycleFilter=Active&modelVersionFilter=All+Runs&datsaetsFilter=

Rocky Linux Rocky Wiki Rocky Forums Rocky Mattermost Rocky Reddit

It looks like you haven't started Firefox in a while. Do you want to clean it up for a fresh, like-new experience? And by the way, welcome back! Refresh Firefox...

mlflow 2.14.1 Experiments Models

Experiments

Search Experiments

Default

cartvista-churn

cartvista-churn Provide Feedback Add Description

metrics.rmse < 1 and params.model = "tree"

Time created State: Active Datasets Sort: Created Columns Group by

Table Chart Evaluation Experimental Traces Experimental

Run Name	Created	Dataset	Duration	Source	Models	test_accuracy	test_auc	test_f1	Parameters
rf-run	1 minute ago	-	10.7s	train_tr...	sklearn	0.9056	0.67452718879904...	0.0923076...	e37ce0cb99
rf-run	1 minute ago	-	13.3s	train_tr...	sklearn	0.9064	0.68120337052375...	0.0330578...	e37ce0cb99
rf-run	1 minute ago	-	8.4s	train_tr...	sklearn	0.9064	0.70012296225888...	0	e37ce0cb99
logreg-run	2 minutes ago	-	10.9s	train_tr...	sklearn	0.908	0.74957189520296...	0.0495867...	e37ce0cb99
logreg-run	2 minutes ago	-	10.2s	train_tr...	sklearn	0.908	0.74957189520296...	0.0495867...	e37ce0cb99
logreg-run	2 minutes ago	-	15.9s	train_tr...	sklearn	0.908	0.74877226333537...	0.0495867...	e37ce0cb99

6 matching runs

Run Name	Created	Duration	Source	Models	recall_score	test_accuracy	test_auc	test_f1	test_precision	data_fingerprint
rf-run	20 minutes ago	10.7s	train_tr...	sklearn	-	0.9056	0.6745271...	0.0923076...	-	e37ce0b99
rf-run	20 minutes ago	13.3s	train_tr...	sklearn	-	0.9064	0.6812033...	0.0330578...	-	e37ce0b99
rf-run	20 minutes ago	8.4s	train_tr...	sklearn	-	0.9064	0.7001229...	0	-	e37ce0b99
logreg-run	1 minute ago	29.8s	train_tr...	sklearn	0.0256410...	0.908	0.7495718...	0.0495867...	0.75	e37ce0b99
logreg-run	21 minutes ago	10.9s	train_tr...	sklearn	-	0.908	0.7495718...	0.0495867...	-	e37ce0b99
logreg-run	21 minutes ago	10.2s	train_tr...	cartvista-churn v1	-	0.908	0.7495718...	0.0495867...	-	e37ce0b99

Run Name	Created	Duration	Source	Models	recall_score	test_accuracy	test_auc	test_f1	test_precision	data_fingerprint
rf-run	42 minutes ago	10.7s	train_tr...	sklearn	-	0.9056	0.6745271...	0.0923076...	-	e37ce0b99
rf-run	42 minutes ago	13.3s	train_tr...	sklearn	-	0.9064	0.6812033...	0.0330578...	-	e37ce0b99
rf-run	42 minutes ago	8.4s	train_tr...	sklearn	-	0.9064	0.7001229...	0	-	e37ce0b99
logreg-run	23 minutes ago	29.8s	train_tr...	cartvista-churn v2	0.0256410...	0.908	0.7495718...	0.0495867...	0.75	e37ce0b99
logreg-run	43 minutes ago	10.9s	train_tr...	sklearn	-	0.908	0.7495718...	0.0495867...	-	e37ce0b99
logreg-run	43 minutes ago	10.2s	train_tr...	cartvista-churn v1	-	0.908	0.7495718...	0.0495867...	-	e37ce0b99
logreg-run	43 minutes ago	15.9s	train_tr...	sklearn	-	0.908	0.7487722...	0.0495867...	-	e37ce0b99
gb-run	3 minutes ago	14.2s	train_tr...	sklearn	0.0854700...	0.9104	0.6912478...	0.1515151...	0.6666666...	e37ce0b99
gb-run	4 minutes ago	28.0s	train_tr...	sklearn	0.0683760...	0.9072	0.6948235...	0.1212121...	0.5333333...	e37ce0b99
gb-run	6 minutes ago	0.8s	train_tr...	-	-	-	-	-	-	e37ce0b99

10 matching runs

```
(.venv) (base) s2025ct05032@ip-10-3-26-18:~/workspace/cartvista-mlops$ python consume_model.py
/home/s2025ct05032/workspace/cartvista-mlops/.venv/lib/python3.12/site-packages/mlflow/utils/requirements_u
tils.py:20: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pk
g_resources.html. The pkg_resources package is slated for removal as early as 2025-11-30. Refrain from usin
g this package or pin to Setuptools<81.
import pkg_resources # noqa: TID251
customer_000 -> churn prediction: 0
customer_001 -> churn prediction: 0
customer_002 -> churn prediction: 0
customer_003 -> churn prediction: 0
customer_004 -> churn prediction: 0
```

Complete E1–E4 (required); E5 is a stretch goal. Record answers and screenshots in your submission document.

E1 — Close the metric gap (10 min)

In Step 2's checkpoint you explained the accuracy-vs-AUC gap. Now act on it: add precision and recall to the metrics logged in `train_tracked.py` and re-run the best configuration. Which of the two is more important for CartVista's retention team, whose offers cost money to send? One paragraph.

Ans.

```
terminal
python train_no_tracking.py
AUC=0.7001 accuracy=0.9064
```

The reason accuracy is high while AUC is much lower is class imbalance. In Step 1, the dataset has a churn rate of around 8–10% (the expected value is 8.42%), meaning roughly 90% of customers do not churn.

If a model simply predicted “no churn” for every customer, it would already achieve roughly 90% accuracy, despite identifying zero churners. That's why accuracy can look impressive here without indicating that the model is good at distinguishing churners from non-churners. AUC, on the other hand, evaluates how well the model separates the two classes across different classification thresholds, so an AUC of 0.7001 indicates only moderate discrimination. Thus, the 91% accuracy is largely driven by the majority non-churn class, while the ~0.70 AUC gives a more meaningful picture of the model's ability to distinguish customers who will churn from those who will not.

E2 — A third challenger (15 min)

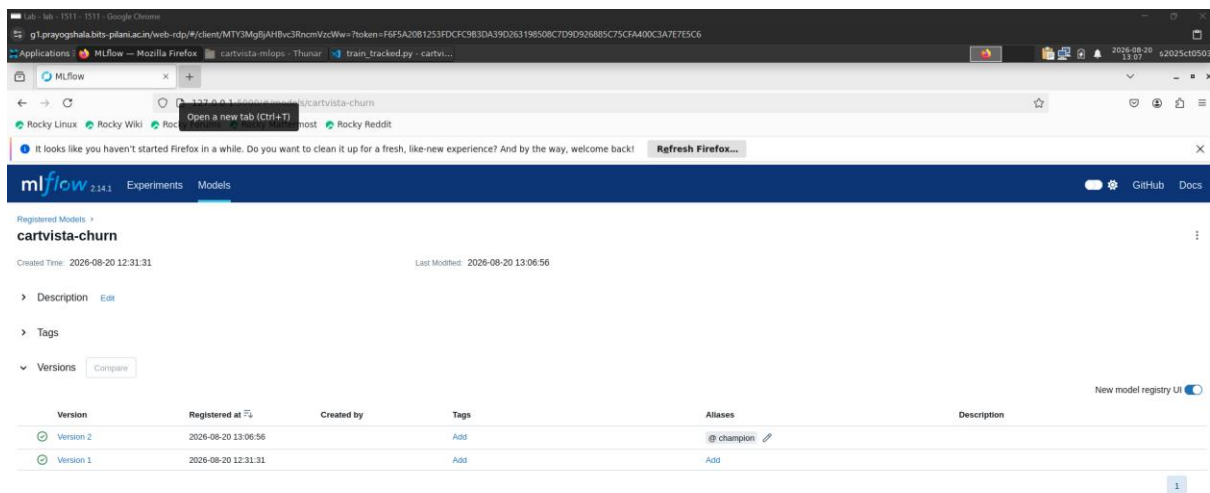
Extend `build_model()` with kind “gb” (sklearn's `GradientBoostingClassifier`, expose `learning_rate` and `n_estimators`). Run two configurations. Does the champion change when you re-run `register_model.py`? Screenshot the Models page showing the version history.

Ans.

```
if kind == "gb":
```

```
    return GradientBoostingClassifier(learning_rate=args.learning_rate,
                                     n_estimators=args.n_estimators, random_state=42)
```

For E2, I extended `build_model()` with `GradientBoostingClassifier` and exposed `learning_rate` and `n_estimators`, then ran two Gradient Boosting configurations. The two new runs achieved AUCs of approximately 0.6912 and 0.6948, both lower than the existing best Random Forest run with $AUC \approx 0.7561$. Therefore, after re-running `register_model.py`, the champion does not change: the Random Forest configuration (`n_estimators=200`, `max_depth=8`) remains the best model because the registry script selects the run with the highest `test_auc`. The screenshot shows the two new gb-run entries and confirms that their AUCs are below the existing RF runs; the model version history should therefore retain the previous champion while a new version is created only if the registration script registers the same best run again.



E3 — Tags for humans (10 min)

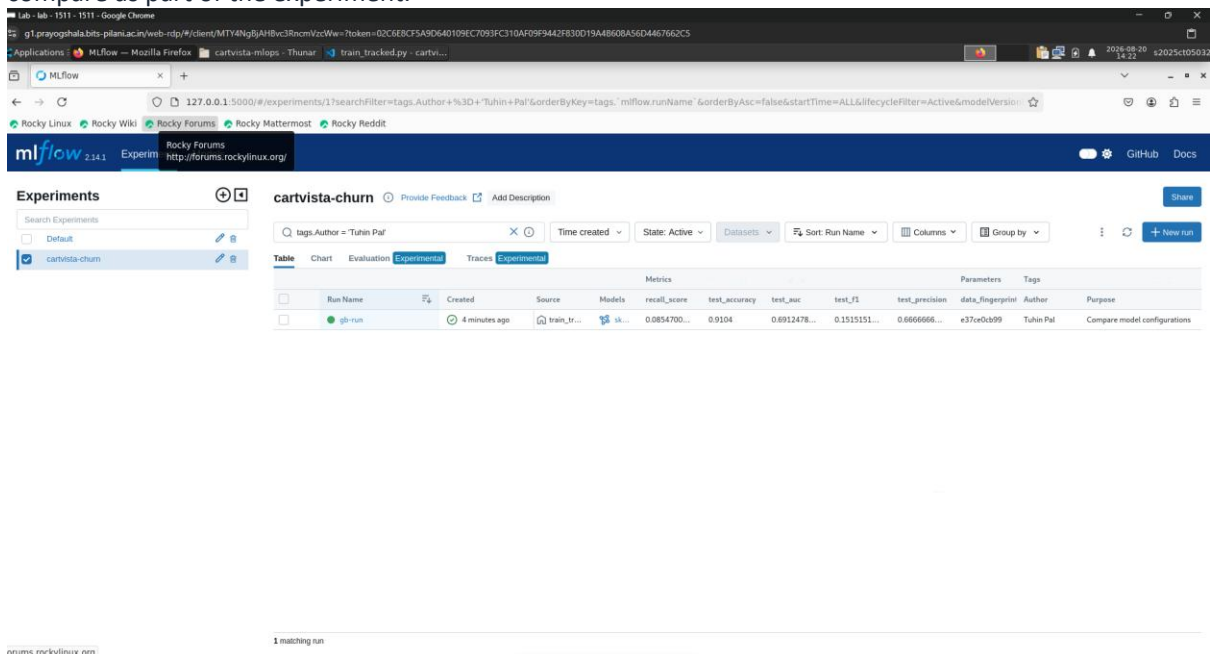
Add `mlflow.set_tag("author", "<your-name>")` and a “purpose” tag to each run. Then, in the UI, filter runs to show only yours. Why are tags the right home for this rather than parameters?

Ans.

```
mlflow.set_tag("Author", "Tuhin Pal")
```

```
mlflow.set_tag("Purpose", "Compare model configurations")
```

tags are the right place because they store human/contextual metadata about a run, such as who created it (Author) and why it was created (Purpose). Parameters, on the other hand, describe the technical configuration used to train the model, such as `n_estimators`, `max_depth`, or `C`, which are values we may compare as part of the experiment.



E4 — Break it, observe, explain (10 min)

Delete one row from churn_data.csv (keep a backup!) and run one more experiment. Compare its data_fingerprint with earlier runs in the UI. In two sentences: what real Priya-vs-Arjun dispute does this one logged parameter prevent? Restore the backup afterwards.

Ans.

The data_fingerprint parameter prevents a real Priya-vs-Arjun dispute about whether two model results were produced using the same dataset version. If their AUC scores differ, the fingerprint lets the team verify whether they trained on the same exact churn_data.csv snapshot or whether the underlying data had changed, making the model comparison fair and reproducible.

Run Name	Created	Duration	Source	Models	recall_score	test_accuracy	test_auc	test_f1	test_precision	data_fingerprint	Author
gb-run	1 minute ago	29.5s	train_tr...	sklearn	0.0683760...	0.9304	0.7154215...	0.125	0.7272727...	6a0c42725	Tuhin Pal
gb-run	12 minutes ago	30.8s	train_tr...	sklearn	0.0854700...	0.9304	0.6912478...	0.1515151...	0.6666666...	e37ce0b599	Tuhin Pal

E5 (stretch) — Autolog vs manual (20 min)

Create train_auto.py using mlflow.sklearn.autolog() and no manual log calls. Compare what it captures against your manual runs. List one thing autolog captured that you did not, and one thing your manual script logs that autolog cannot know.

Ans.

```
train_auto.py
"""CartVista churn training WITH MLflow autologging."""
import argparse
import mlflow
import mlflow.sklearn
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

TRACKING_URI = "sqlite:///mlflow.db"
EXPERIMENT = "cartvista-churn"

def build_model(kind, args):
    if kind == "logreg":
        return LogisticRegression(C=args.C, max_iter=1000)

    if kind == "rf":
```

```

        return RandomForestClassifier(
            n_estimators=args.n_estimators,
            max_depth=args.max_depth,
            random_state=42
        )

    if kind == "gb":
        return GradientBoostingClassifier(
            learning_rate=args.learning_rate,
            n_estimators=args.n_estimators,
            random_state=42
        )

    raise ValueError(f"unknown model kind: {kind}")

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--model", choices=["logreg", "rf", "gb"], required=True)
    ap.add_argument("--C", type=float, default=1.0)
    ap.add_argument("--n-estimators", type=int, default=100)
    ap.add_argument("--max-depth", type=int, default=6)
    ap.add_argument("--learning-rate", type=float, default=0.1)
    args = ap.parse_args()

    mlflow.set_tracking_uri(TRACKING_URI)
    mlflow.set_experiment(EXPERIMENT)

    # Automatic logging: no manual log_param, log_metric, or log_model calls.
    mlflow.sklearn.autolog()

    df = pd.read_csv("churn_data.csv")
    X = df.drop(columns=["churn"])
    y = df["churn"]

    X_tr, X_te, y_tr, y_te = train_test_split(
        X, y,
        test_size=0.25,
        random_state=42,
        stratify=y
    )

    model = build_model(args.model, args)
    model.fit(X_tr, y_tr)

    print("Autologged run successfully.")

if __name__ == "__main__":
    main()

```

terminal

```

python train_auto.py --model gb --learning-rate 0.05 --n-estimators 200
2026/08/20 14:39:18 WARNING mlflow.utils.autologging_utils: You are using an unsupported
version of sklearn. If you encounter errors during autologging, try upgrading / downgrading
sklearn to a supported version, or try upgrading MLflow.
2026/08/20 14:39:23 INFO mlflow.utils.autologging_utils: Created MLflow autologging run with
ID '8251d7fe32de47a1bdc6e7bd355c53bc', which will track hyperparameters, performance
metrics, model artifacts, and lineage information for the current sklearn workflow
2026/08/20 14:39:23 WARNING mlflow.utils.autologging_utils: MLflow autologging encountered a
warning: "/home/s2025ct05032/workspace/cartvista-mlops/.venv/lib/python3.12/site-
packages/mlflow/types/utils.py:406: UserWarning: Hint: Inferred schema contains integer
column(s). Integer columns in Python cannot represent missing values. If your input data
contains missing values at inference time, it will be encoded as floats and will cause a
schema enforcement error. The best way to avoid this problem is to infer the model schema
based on a realistic data sample (training dataset) that includes missing values.
Alternatively, you can declare integer columns as doubles (float64) whenever these columns
may have missing values. See `Handling Integers With Missing Values

```

```
<https://www.mlflow.org/docs/latest/models.html#handling-integers-with-missing-values>`_ for more details."
2026/08/20 14:39:31 WARNING mlflow.utils.autologging_utils: MLflow autologging encountered a warning: "/home/s2025ct05032/workspace/cartvista-mlops/.venv/lib/python3.12/site-packages/mlflow/types/utils.py:406: UserWarning: Hint: Inferred schema contains integer column(s). Integer columns in Python cannot represent missing values. If your input data contains missing values at inference time, it will be encoded as floats and will cause a schema enforcement error. The best way to avoid this problem is to infer the model schema based on a realistic data sample (training dataset) that includes missing values. Alternatively, you can declare integer columns as doubles (float64) whenever these columns may have missing values. See `Handling Integers With Missing Values`_ for more details."
Autologged run successfully.
```

Run Name	Created	Duration	Source	recall_score	test_accuracy	test_auc	test_f1	test_precision	training_accuracy	training_f1_score	training_log_loss	train
colorful-bird-673	14 minutes ago	33.1s	train_auto.py	-	-	-	-	-	0.9178666...	0.8903105...	0.2252778...	0.91
gb-run	27 minutes ago	29.5s	train_tracked.py	0.0683760...	0.9104	0.7154215...	0.125	0.7272727...	-	-	-	-
gb-run	38 minutes ago	30.8s	train_tracked.py	0.0854700...	0.9104	0.6912478...	0.1515151...	0.6666666...	-	-	-	-
gb-run	1 hour ago	14.2s	train_tracked.py	0.0854700...	0.9104	0.6912478...	0.1515151...	0.6666666...	-	-	-	-
gb-run	1 hour ago	28.0s	train_tracked.py	0.0683760...	0.9072	0.6948235...	0.1212121...	0.5333333...	-	-	-	-
gb-run	1 hour ago	0.8s	train_tracked.py	-	-	-	-	-	-	-	-	-
logreg-run	2 hours ago	29.8s	train_tracked.py	0.0256410...	0.908	0.7495718...	0.0495867...	0.75	-	-	-	-
rf-run	2 hours ago	10.7s	train_tracked.py	-	0.9056	0.6745271...	0.0923076...	-	-	-	-	-
rf-run	2 hours ago	13.3s	train_tracked.py	-	0.9064	0.6812033...	0.0330578...	-	-	-	-	-
rf-run	2 hours ago	6.4s	train_tracked.py	-	0.9064	0.7001229...	0	-	-	-	-	-
logreg-run	2 hours ago	10.9s	train_tracked.py	-	0.908	0.7495718...	0.0495867...	-	-	-	-	-
logreg-run	2 hours ago	10.2s	train_tracked.py	-	0.908	0.7495718...	0.0495867...	-	-	-	-	-
logreg-run	2 hours ago	15.9s	train_tracked.py	-	0.908	0.7487722...	0.0495867...	-	-	-	-	-

Autolog captured training/model metrics and sklearn metadata automatically that my manual script did not, while my manual script explicitly logged the test-set metrics: AUC, accuracy, F1, precision, and recall. My manual script also logs data_fingerprint, which autolog cannot know because it is a custom identifier for the exact dataset used.

- One reflection paragraph: “What breaks in this setup when Priya and Arjun each run MLflow on their own laptops?” — We pick this thread up in Session 6.

Ans.

If Priya and Arjun each run MLflow on their own laptops, their experiments would be recorded in separate local MLflow stores, so neither person would automatically see or be able to compare the other’s runs, parameters, metrics, artifacts, or model versions. This breaks the shared “team memory” that makes experiments comparable and reproducible, and it would also make the model registry and the single `@champion` model difficult to manage consistently across the team. A shared MLflow tracking and registry setup is therefore needed so that everyone works with the same experiment history and a single source of truth for the model.